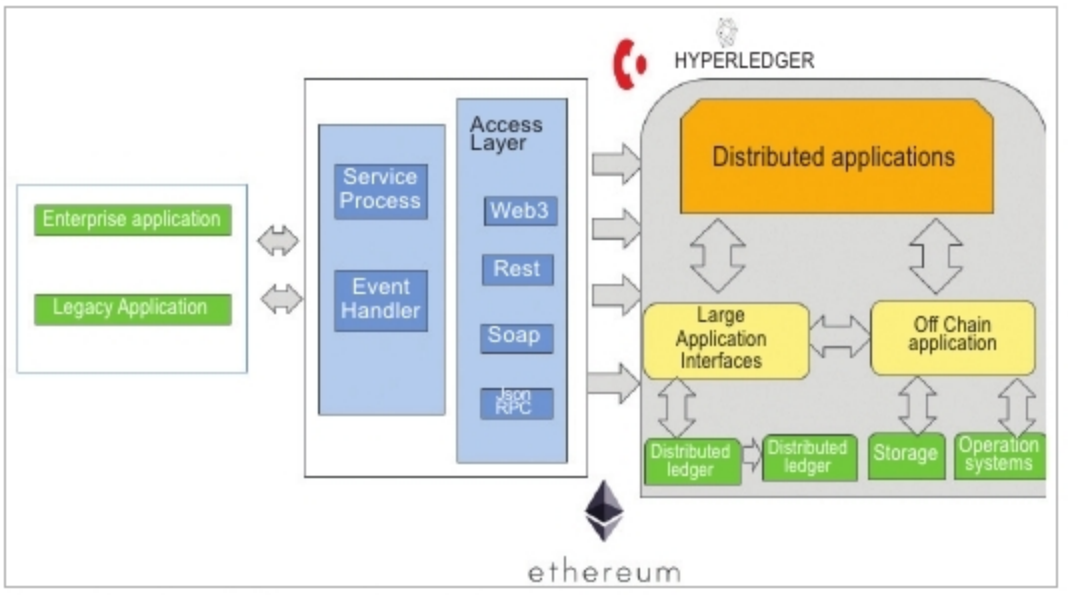

Figure 1: Distributed architecture for blockchain applications

- Performance testing
- API testing
- Functional testing
- CI/CD for blockchain
- Compliance and security testing

Areas of testing required for blockchain applications

The core area of testing for blockchain applications is the external interface to the chain, and this is done through API level testing. Then there is chain level testing for fault tolerance, node level testing such as smart contract testing, and the testing of overall non-functional aspects. Let us look at how we can fulfil each of these requirements.

API testing for blockchain applications

In blockchain applications, the application programming interface (API) plays a major role in defining the whole transaction process, the issue and receipt of payment transactions for virtual currencies, and the corresponding data management.

External applications access a blockchain network using its exposed API interface. Smart contracts are invoked by the external applications using these API interfaces. Hence, it's important to validate this interface to ensure seamless integration.

Peer/node and fault-tolerant testing

Since the blockchain involves a peer-to-peer architecture, it is extremely important to validate the encrypted and decrypted data transmission process to ensure it is working flawlessly so that no data is lost.

One of the main goals of peer-to-peer node testing is to ensure the consistency of transactions that are stored in a proper sequence across all the nodes. This means consistency checks should be carried out to maintain all nodes under normal conditions as well as in various scenarios in which the nodes fail concurrently. Typical areas of testing involve consistency checks with the database, synchronisation of blocks, node restarts, node rejoining, network failure, etc. The ultimate goal of this testing is to ensure consensus is achieved across all nodes in the order in which transactions take place. Consistency and sequences are the key parts of testing.

Smart contracts

A smart contract is a set of rules in the form of programmable constructs. These rules are capable of automatically enforcing themselves when predefined conditions are met. Testing smart contracts is extremely critical as once the contract is deployed, it can never be changed. Testing smart contracts involves the following:

- Simulation of all possible expected and unexpected scenarios and conditions of each and every contract
- API testing with all boundary and conditional statements for individual contracts, and also integration testing
- Validation of the encryption for the distributed ledger
- Testing of all possible scenarios that could result from applying different business logic
- Testing the triggers and execution of transactions
- Real-time auditing of transactions

More importantly, as all the above testing happens dynamically in a large number of nodes, it can be done only through automated testing.

Automated testing

There are many tools that enable automation testing in various blockchain platforms. A few of them are listed below.

1) The most important tools that perform automation testing for the applications built on the Ethereum platform are Ethereum Tester, Truffle, Test RPC, Populus and Manticore.
2) Hyper Ledger Composer can be used for testing HyperLedger. It can be integrated with the Mocha/Chai test framework.
3) Corda provides a lot of inbuilt capabilities that can be used to build automation tests for applications based on this platform.

Non-functional testing

Testing for performance, compliance and security are the main non-functional aspects that need to be covered for blockchains and their applications.

Performance validation: Due to high throughput and performance requirements, blockchain data is stored in key-value pairs in the database using hashing algorithms. The blockchain network needs to be tested for performance and latency, which would vary, based on:

- The size of the network
- The size of the block
- The expected size of transactions
- The consensus protocol used